

## 9 - System design

Il System design è l'**identificazione** dei principali componenti e delle relazioni tra i componenti stessi per quanto riguarda l'**architettura software**.

Un architettura software è definita per gli stili di **progettazione ad alto livello** e differisce sostanzialmente dall'**Object Oriented design**, che è incentrato sulla **progettazione di dettaglio**.

### Architettura software

#### Definizione

L'architettura software definisce un modello di come il sistema è strutturato e di come i sottosistemi presenti comunicano tra di loro.

Generalmente un sottosistema include le classi, operazioni interne e altri aspetti del sistema che sono strettamente correlati tra i vari sottosistemi.

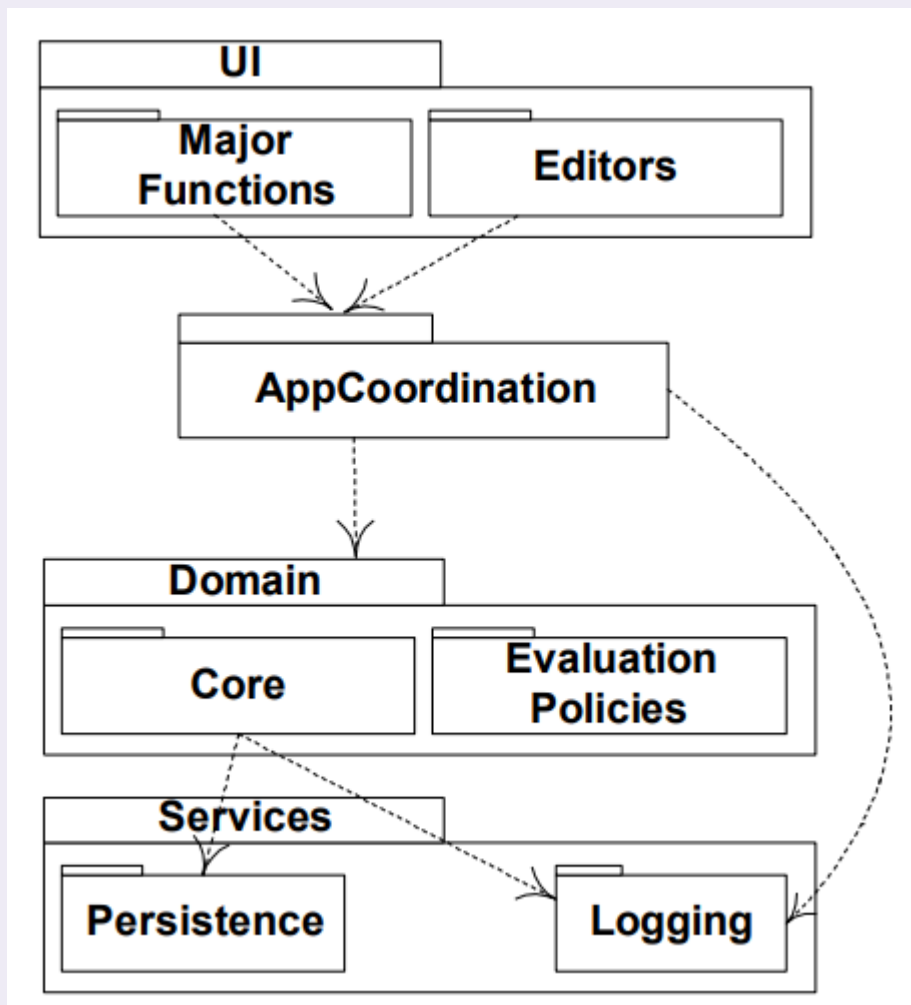
L'architettura **evidenzia le decisioni** che avranno un impatto significativo sui lavori successivi.

### Rappresentazioni

L'architettura è rappresentabile sotto diversi stili dell'UML:

#### Diagramma dei package

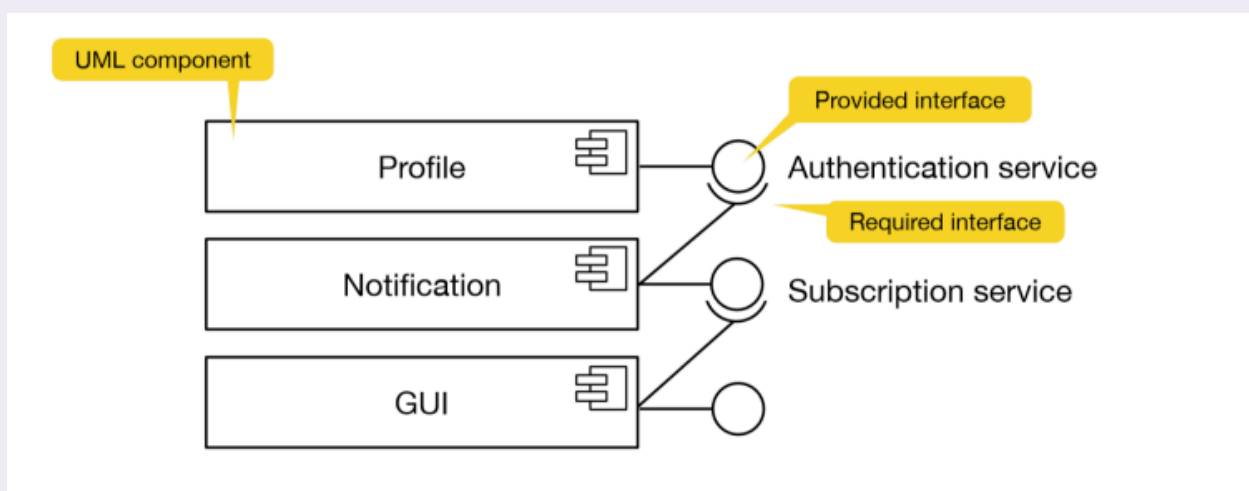
#### Esempio di diagramma dei package



Nell'immagine possiamo notare come elementi di un package (**Services**) richiedono elementi di un altro package che funge da fornitore (**Domain**).

## Diagramma dei componenti

### ≡ Esempio di diagramma dei componenti



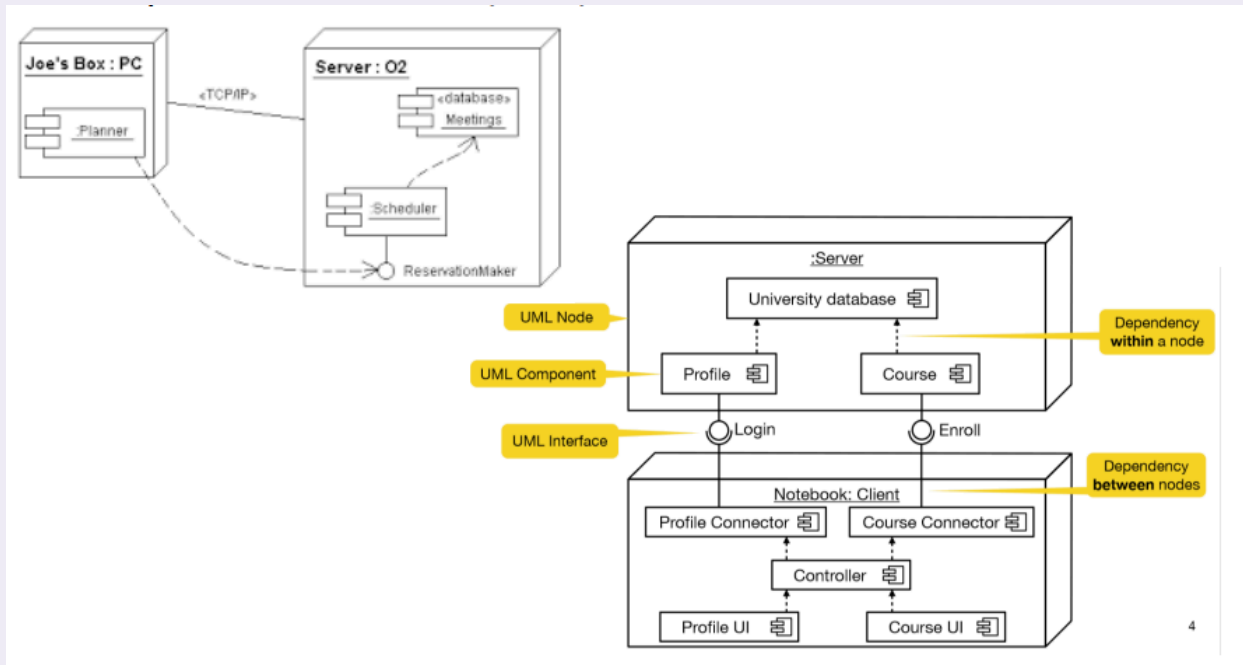
Nel diagramma illustrato si comprende a pieno lo stile di rappresentazione delle obbligazioni e dei collegamenti tra componenti tramite le dipendenze.

Il diagramma dei componenti ci permette di comprendere la modellazione del sistema in termini di **componenti** e **dipendenze tra componenti**.

Un componente generalmente è un'**interfaccia** pubblica, come un'**API**, la dipendenza è un connettore tra il **componente cliente** e il **componente fornitore**.

## Diagramma di deployment

### ≡ Esempio di diagramma di deployment



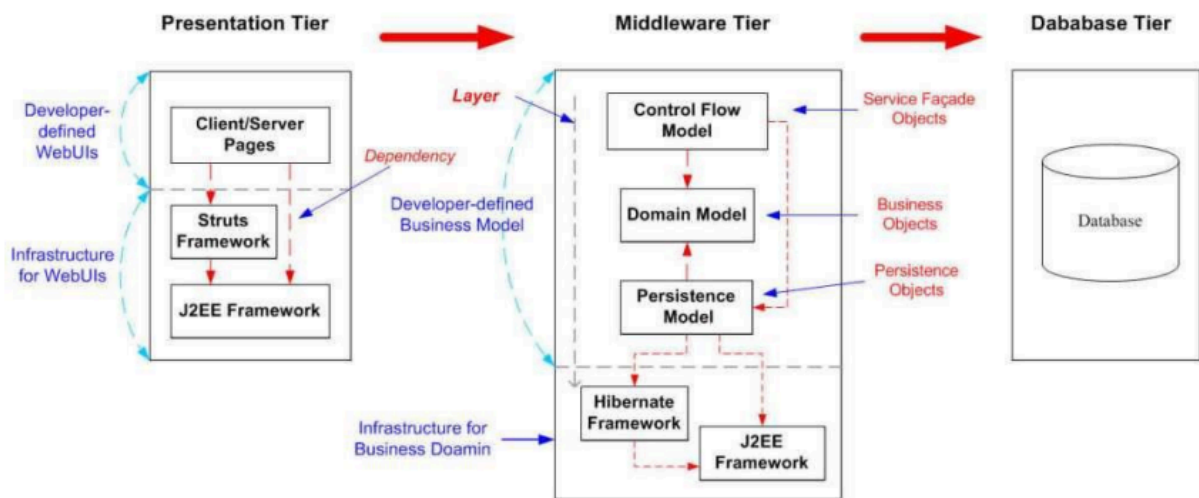
Questo ci permette di avere una rappresentazione grafica della distribuzione dei sottosistemi tra i **nod**i (i componenti hardware).

## Rappresentazione non-UML dell'architettura

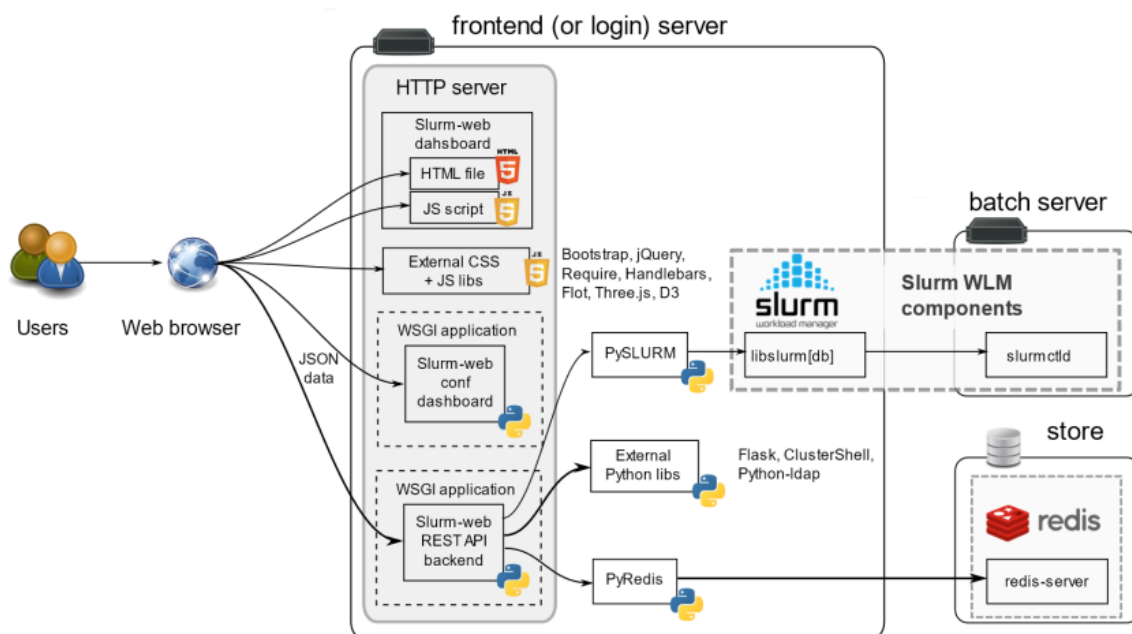
L'architettura in fenomeni aziendali concreti richiede rappresentazioni che non sono riconosciute dall'UML, come:

- **Architettura a 3 livelli**, la quale ci permette di comprendere come lavora il progetto reale nei tre strati logici e fisici, partendo dall'Interfaccia web, Logica di business e

## Database Tier.

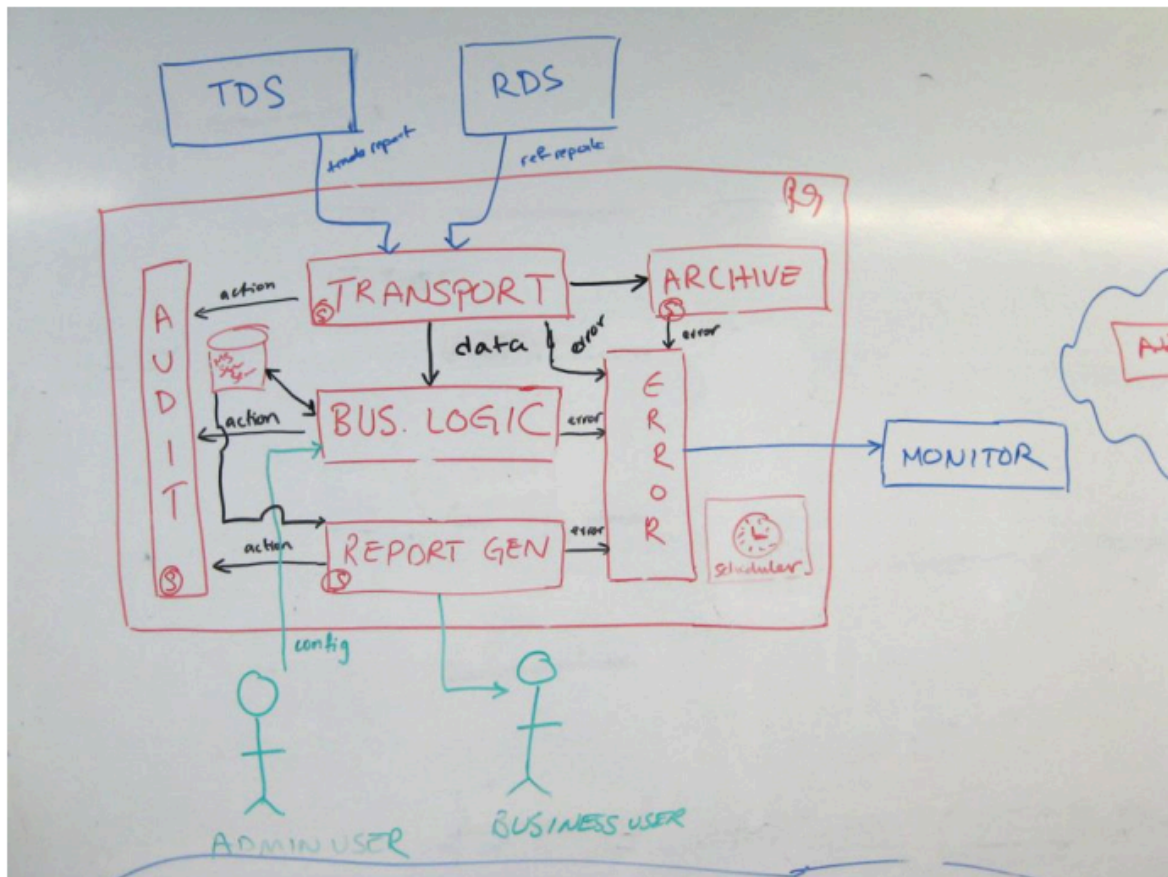


- **Slurm** è una metodologia di rappresentazione di una reale architettura con i vari server a lavoro. Generalmente è usata per comprendere come le diverse tecnologie usate (come HTML, Python, JSON) comunicano tra loro



- **Whiteboard**, usato nelle aziende per creare una bozza iniziale dell'architettura quando il lavoro sta per nascere o c'è bisogno di ricrearlo per capire come aggiustarlo o

migliorarlo.



## Checklist per la revisione di diagrammi di architettura software

La checklist per la revisione dei diagrammi è una **tabella** divisa in tre campi con delle domande al suo interno. Un diagramma per essere corretto e adatto alla comprensione **deve rispondere in maniera affermativa** a (quasi) tutte le domande proposte al suo interno.

I campi presenti nella tabella sono:

- **General:** le domande principali da porre sono:

|                                              |     |    |
|----------------------------------------------|-----|----|
| Does the diagram have a title?               | Yes | No |
| Do you understand what the diagram type is?  | Yes | No |
| Do you understand what the diagram scope is? | Yes | No |
| Does the diagram have a key/legend?          | Yes | No |

- **Elements:** hanno molte domande, tra cui alcune sono caratteristiche proprio di singoli elementi, per cui generalmente si domanda se ogni element ha un nome e un tipo, se i colori e le forme hanno un particolare significato e se sono spiegati in modo chiaro.

- **Relationships:** le domande a livello comunicativo tra team ed interni sono le seguenti:

|                                                                                                                                                 |     |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-----|----|
| Does every line have a label describing the intent of that relationship?                                                                        | Yes | No |
| Where applicable, do you understand the technology choices associated with every relationship? (e.g. protocols for inter-process communication) | Yes | No |
| Do you understand the meaning of all acronyms and abbreviations used?                                                                           | Yes | No |
| Do you understand the meaning of all colours used?                                                                                              | Yes | No |
| Do you understand the meaning of all arrow heads used?                                                                                          | Yes | No |
| Do you understand the meaning of all line styles used? (e.g. solid, dashed, etc)                                                                | Yes | No |

## Progettazione per il cambiamento

Un'architettura software ben costruita deve essere **modificabile senza problemi nel tempo**.

La maggior parte delle applicazioni ha costanti aggiornamenti, portando quindi alla modifica dei diagrammi UML in maniera costante nel tempo.

Esistono quattro regole fondamentali che un'architettura software deve mantenere nel tempo per poter essere modellata:

- **Information hiding**
- **Obiettivo di alta coesione**
- **Obiettivo di basso accoppiamento**
- **Presentazione separata**

## Principio di information hiding

### Principio di information hiding

Ogni componente deve custodire dei segreti al proprio interno. Se la decisione cambia, solo il componente interessato sarà modificato.

Questo principio si declina nel seguente modo:

- Per i **sottosistemi** la loro interfaccia è pubblica ma l'implementazione è nascosta all'esterno
- Per i **package** le classi sono tutte private, si rendono pubbliche solo le classi che per motivi implementativi devono esserlo
- Per le **classi** le operazioni, funzioni e variabili di istanza sono tutte private, per favorire l'**incapsulamento dei dati**. Esistono casi in cui diviene strettamente necessario rendere pubbliche alcune operazioni, ma non è la norma

## Obiettivo di alta coesione

### Definizione di coesione

La **coesione** misura il grado di dipendenza tra elementi di uno stesso componente, che sia sottosistema o classe

Un componente ad **alta coesione** è un sottosistema o classe con regole chiare e ben definite con una grande responsabilità progettuale.

Viceversa un componente a **bassa coesione** si occupa di casi singoli, irripetibili, fini a se stesso e non riutilizzabili e per questo **molto difficili nella modifica**.

L'obiettivo dell'alta coesione è la necessità di cercare, quanto meno, di **assegnare le responsabilità** in modo tale da ottenere componenti sempre ben definiti.

Per **risolvere** una bassa coesione basta semplicemente delegare le responsabilità di quella classe ad altri componenti.

## Obiettivo di basso accoppiamento

### Definizione di accoppiamento

L'**accoppiamento** misura il grado di dipendenza tra componenti diversi

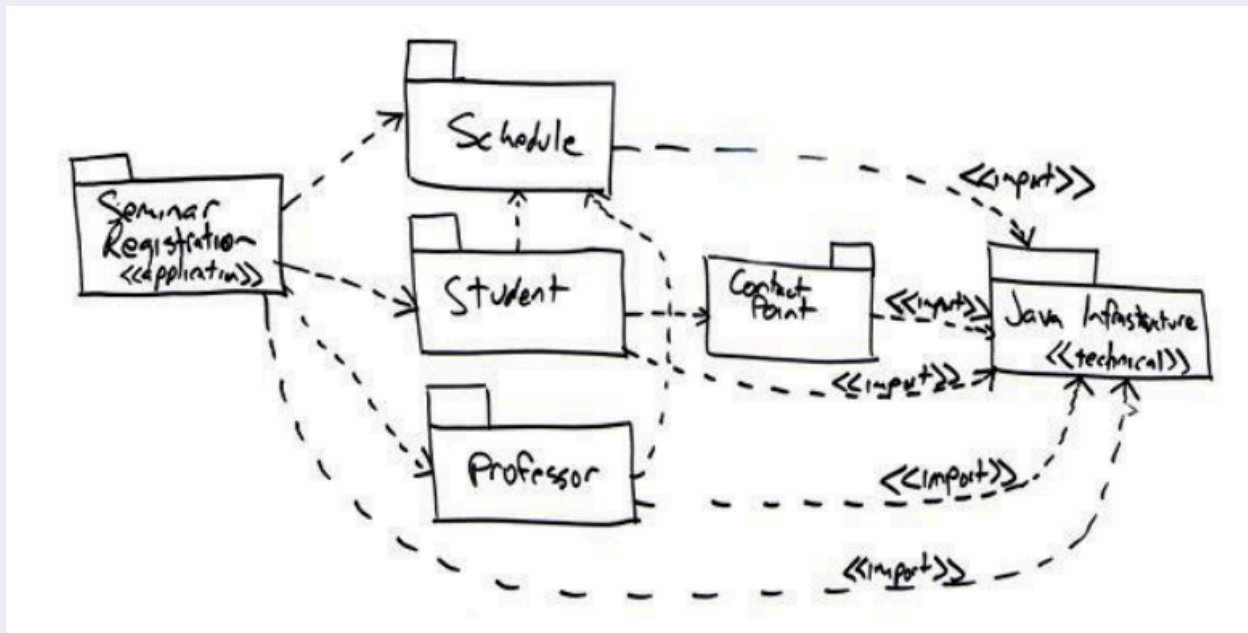
La comunicazione tra diversi sottosistemi o classi è doverosa nei grandi progetti, per questo un **alto accoppiamento** tra i componenti comporta una modifica a cascata inevitabile in caso volessimo modificare un singolo componente.

Viceversa un **basso accoppiamento** permette di eseguire cambiamenti ad un singolo componente senza il rischio di propagarsi agli altri.

Una corretta applicazione dell'**information hiding** comporta già un basso accoppiamento.

Possiamo notare il rispetto di questo obiettivo tramite il diagramma dei package, dove un buon diagramma deve essere **aciclico**, ovvero il grafo delle dipendenze non deve contenere cicli

### Esempio di diagramma aciclico



## Presentazione separata

Questo ultimo principio spiega un'applicazione pratica del basso accoppiamento, ovvero è una buona pratica tenere separata la **logica visiva** (quello che vede l'utente) dalla **logica di dominio** (il vero funzionamento del programma).

E' più facile capire errori e modifiche da adoperare se queste due logiche sono divise tra loro, inoltre:

- è possibile esporre il funzionamento del programma come **API** o **servizio** (operazioni pubbliche).
- è possibile creare interfacce diverse (es. un'app per smartphone e un sito web) usando lo stesso "motore", senza duplicare codice.
- è molto più facile fare **test automatici** sul codice se non c'è l'interfaccia grafica di mezzo.

## Chief Architect

Costruire un programma tramite l'assistenza di assistenti AI è ormai alla portata di tutti; un ottimo **architetto software** deve saper creare dalla base di partenza un progetto complesso, scalabile e usato da milioni di utenti.

Il Chief Architect necessita della piena padronanza di tre requisiti per poter essere definito tale:

- **Conoscenza** delle tecnologie
- **Esperienza** sul campo
- **Creatività** per trovare soluzione a problemi complessi

## Stili architetturali



Un modello di architettura può essere utilizzato come base per il System Design. L'architetto aggiungerà i dettagli alla bozza in base alla specifica dei requisiti.

## Pipe and filter

I sottosistemi inviano i dati ricevuti in input come input per i successivi sottosistemi. Un esempio di architettura Pipe and filter è **Unix shell**. Il nome deriva da **Filter** che è il sottosistema, **Pipe** che è l'associazione tra i sottosistemi realizzata dal SO e **Pipeline**, che è la sequenza lineare dei filtri (sottosistemi)

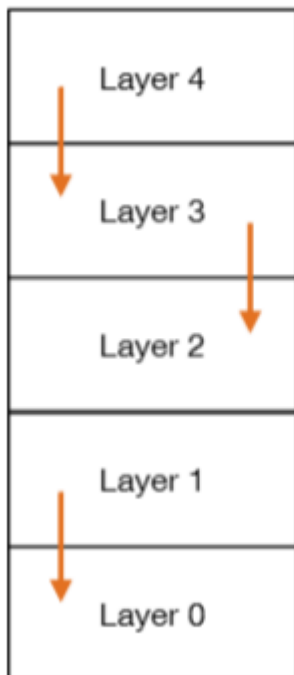
## Layered

È un architettura divisa in **Layer**, dove ogni layer utilizza un'interfaccia richiesta dal **Layer N-1** sopra stante, il quale la detiene o la chiede al precedente  $N - 2$ . Inoltre ogni strato ha il suo livello di astrazione. Questo porta ad un **vantaggio**, ossia le modifiche ai livelli superiori non si propagano a quelli inferiori ma lo **svantaggio** di questa architettura è il calo di prestazioni dato dal grande numero di strati presenti.

La stratificazione si può dividere in **stretta** o **lasca**:

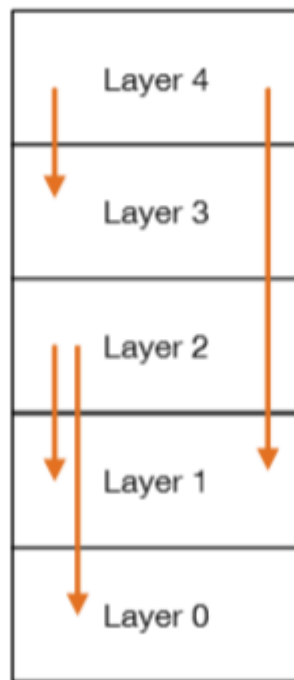
- Quella stretta si basa **Modello ISO/OSI**, in cui lo strato evocabile è solamente quello immediatamente inferiore

### Modello ISO/OSI



- Quella **lasca** si basa sul **Modello TCP/IP**, in cui lo strato è evocabile in qualsiasi livello ci si trova, purché sia comunque uno strato inferiore e non superiore all'attuale.

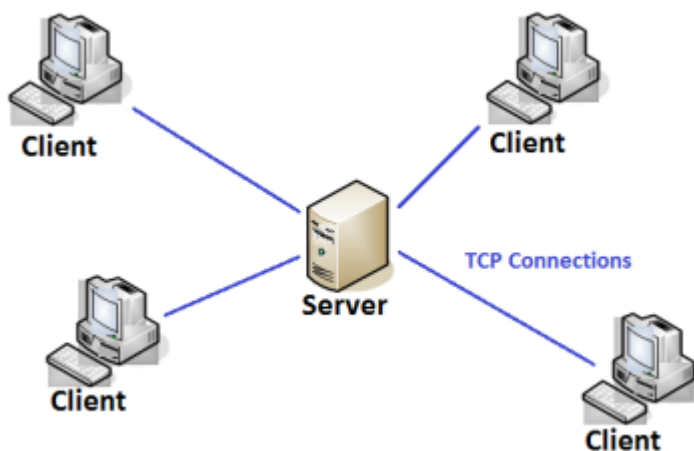
# TCP/IP stack



## Client-Server

Questa architettura è una rivisitazione **distribuita** di quella di tipo layered.

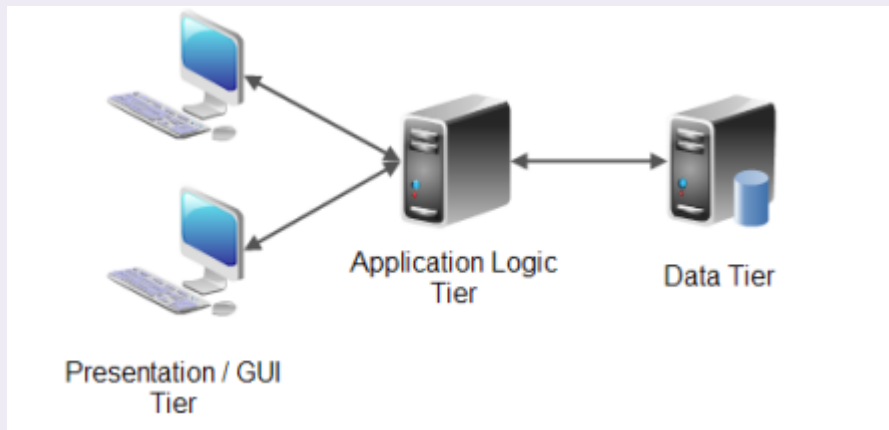
Il client invia al server centrale una **richiesta di comunicazione**, il server è **sempre in ascolto** e appena giunge una richiesta di comunicazione la accetta **eseguendo quella e rispedendo a ritroso le successive** durante il periodo di esecuzione di quella accettata. Il **vincolo** di questa architettura è che non vi è un effettiva comunicazione tra i client ma tutto deve passare dal server per aver un confronto.



## N-tier:

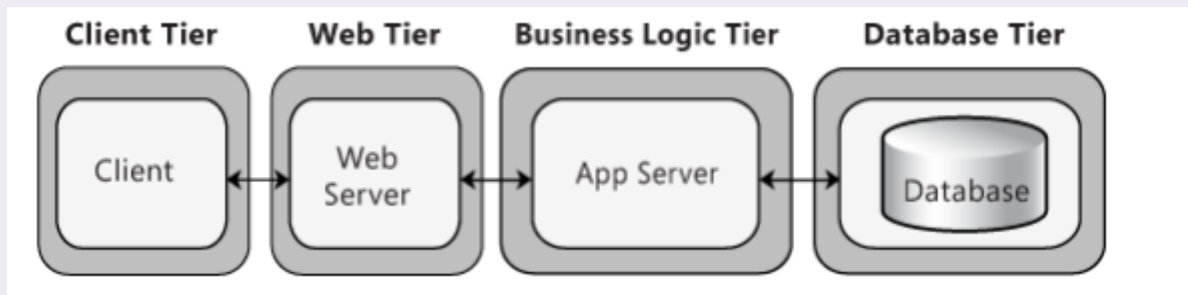
È un ulteriore rivisitazione distribuita dell'architettura layered, dove gli strati sono **localizzati su  $N$  nodi**. Può essere **3-tier** per le applicazioni gestionali, con la seguente architettura:

☰ Esempio di architettura a 3 nodi



### ☰ Esempio di architettura a 4 nodi

Oppure può essere 4-**tier** per le applicazioni web, con la seguente implementazione:



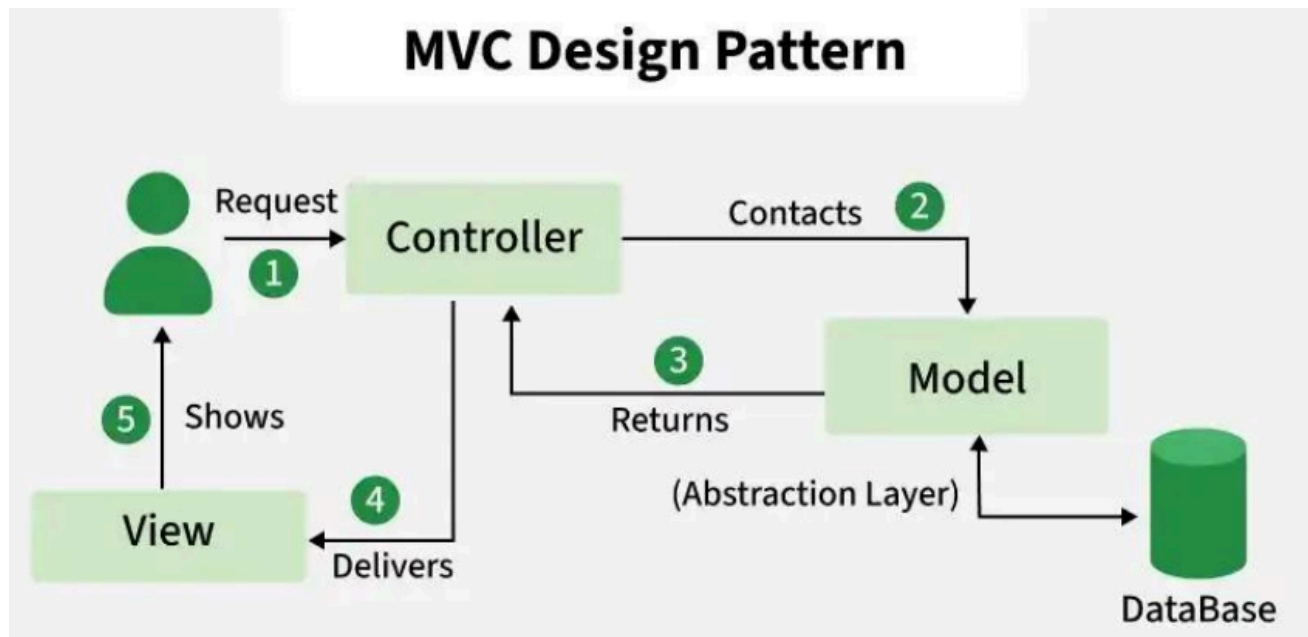
## MCV

I **Model-View-Controller** (MVC) sono architetture utilizzate per i **sistemi interattivi**. Qui la logica dei dati (View) viene separata dalla logica di business (Model).

La parte sul **Model** include la conoscenza del dominio e i metodi per l'accesso ai dati, mentre la parte sul **View** include la rappresentazione visuale dei dati.

Infine esiste anche la classe madre, detta **Controller**, la quale gestisce la sequenza di informazioni con l'utente. Gli input accettati vengono trasformati in comandi o per il Model o

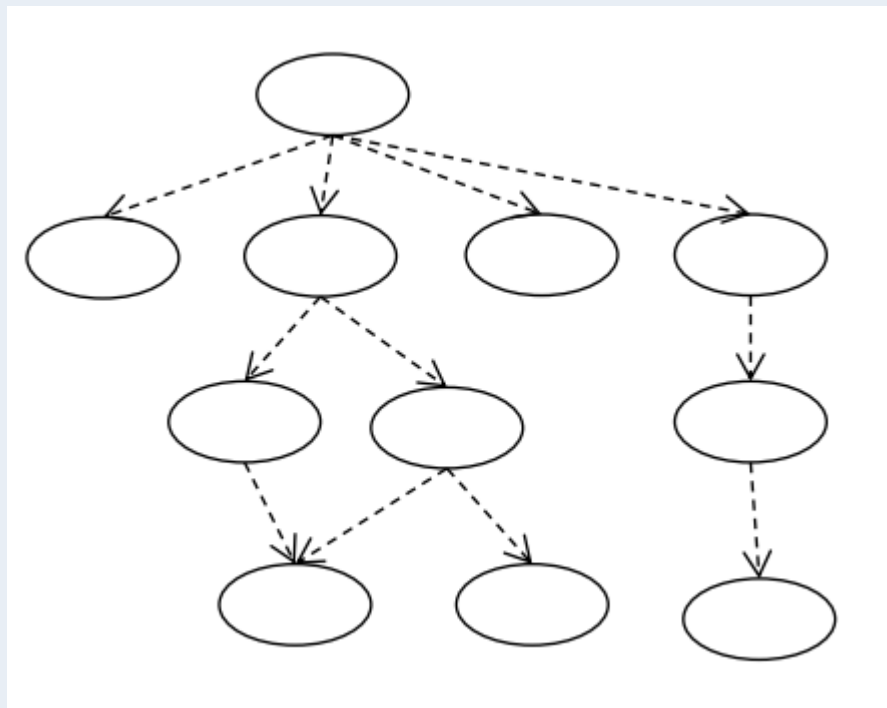
per la View.



## Microservice architecture

### Definizione di microservice

Una **Microservice Architecture** (architettura a microservizi) è un'architettura software basata su una collezione di servizi in cui la funzionalità totale del sistema deriva dalla **composizione di più servizi minori**.



In questa tipologia di architettura, il progetto viene suddiviso in parti più piccole che seguono regole precise:

- Ogni servizio è un'**unità di distribuzione indipendente**.

- Ogni servizio fornisce una **piccola quantità di funzionalità** specifiche (rispettando il principio dell'alta coesione).
- Ogni servizio comunica con gli altri tramite delle **interfacce di servizio**, utilizzando solitamente le **API RESTful**.

## API RESTful

Le comunicazioni in un'architettura a microservizi avvengono prevalentemente tramite **API RESTful**. Queste interfacce sono accessibili sfruttando i metodi standard del protocollo **HTTP**, i quali sono perfettamente equivalenti alle classiche operazioni di database **CRUD** (Create, Read, Update, Delete):

- **POST** (equivalente a CREATE)
- **GET** (equivalente a READ)
- **PUT** (equivalente a UPDATE)
- **DELETE** (equivalente a DELETE)

Questi metodi vengono applicati a una singola risorsa o a una collezione di risorse. Affinché la comunicazione avvenga correttamente, le API RESTful si basano su due concetti fondamentali:

- **Identificazione delle risorse:** le risorse vengono identificate in modo univoco tramite un indirizzo **URI**
- **Stato delle risorse:** lo stato e i dati delle risorse richieste vengono restituiti in un **formato negoziabile** tra client e server. I formati di scambio dati più utilizzati sono il **JSON** e l'**XML**.